

ZESTINE TECHNOLOGIES

Ze AI

Current Code Flow & RAG Implementation Plan

RAG

MCP

C# Terminal

Azure OpenAI

Document Version 2.0

Last Updated: 7 May 2026

ZeManage AI Architecture Documentation

Ze AI — Current Code Flow & RAG Implementation Plan

ZeManage AI Architecture Documentation *Covers: how Ze AI works today, where every piece lives in the code, how RAG fits in, and what it costs on the existing Azure infrastructure.*

Table of Contents

- [1. Part 1 — How Ze AI Works Today](#)
 - [2. Part 2 — Code Walkthrough: Every Step of a User Query](#)
 - [3. Part 3 — Where Each Piece Lives](#)
 - [4. Part 4 — RAG: What It Adds and How It Plugs Into the Current Flow](#)
 - [5. Part 5 — Azure Resources: What You Have Today](#)
 - [6. Part 6 — RAG Cost on Your Existing Azure Plan](#)
 - [7. Part 7 — Implementation Steps Mapped to Your Setup](#)
 - [8. Part 8 — Azure OpenAI: Quotas, Batch Limits, Rate Limits](#)
 - [9. Part 9 — MCP Server: What It Is, What It Costs, Why It Matters](#)
 - [10. Part 10 — Existing C# Terminal Analysis \(RevitPythonShell-Based\)](#)
 - [11. Part 11 — Unified Roadmap: RAG + MCP + Terminal](#)
 - [12. Part 12 — Locked Plan: AI Terminal + MCP \(Phase 3, In Progress\)](#)
-

Part 1 — How Ze AI Works Today

Ze AI is a chat assistant inside the Revit plugin that combines:

- An LLM (OpenAI GPT-4o-mini) for natural language reasoning
- A keyword classifier that decides what kind of question is being asked
- Live model context fetched from the backend (file size, element counts, warnings, alerts)
- Three diagnostic engines for visibility, warnings, and database queries
- A knowledge base of best practices, errors, performance tips, workflows, and product info
- Persistent chat history in SQLite

The user types a message. Behind the scenes, Ze AI assembles a rich system prompt with the right knowledge and live data, sends it to OpenAI, and streams back the answer.

Part 2 — Code Walkthrough: Every Step of a User Query

This is what happens, step by step, when a user types *"why is my model so slow?"* and presses send.

Step 1 — UI Input

File: [BIManage/Views/AI/ZestAiDialog.xaml](#)

The user types into a `TextBox` bound to `UserMessage` on the ViewModel. The "Send" button binds to `SendMessageCommand`.

Step 2 — Command Triggered

File: [BIManage/ViewModels/AI/ZestAiViewModel.cs](#)

`SendMessageCommand` calls `SendMessageAsync()`. This is the orchestration entry point.

```
SendMessageAsync() does:
1. Add the user's message to the Messages collection (shows in chat)
2. Save the user message to SQLite via ChatRepository
3. Set IsThinking = true (animated dots appear)
4. Run the prompt pipeline (steps 3-9 below)
5. Add the AI response to Messages
6. Save the AI response to SQLite
7. Set IsThinking = false
```

Step 3 — Off-Topic / Identity Filter (zero-cost guardrail)

File: `ZestAiViewModel.GetPreformedAnswerIfOffTopic(userText)`

Before any tokens are spent, the message is checked for:

- Identity questions ("Who are you?") → pre-canned reply, no LLM call
- Off-topic questions (politics, weather, recipes) → block + redirect message
- Bypass attempts ("how do I uninstall ZeManage?") → admin contact reply

If matched, the answer is returned immediately. No tokens used. No knowledge fetched.

Step 4 — Intent Classification

File: `BIManage/AI/Knowledge/IntentClassifier.cs`

The query is analyzed by keyword matching to decide what kind of question it is:

Detected Intent	Knowledge Sections Loaded	Special Action
Errors keywords ("crash", "fail")	Errors	None
Best practice keywords ("standards", "naming")	BestPractices	None
Performance keywords ("slow", "purge", "optimize")	Performance	None
Workflow keywords ("how to", "export")	Workflows	None
Product keywords ("ZeManage", "protection mode")	Product	None
Visibility query (regex matches "not visible", element ID)	None	Run <code>ElementVisibilityService</code>
Warning query ("warnings", "resolve warning")	None	Run <code>WarningResolutionService</code>
Local DB query ("sync history", "audit log")	None	Run <code>LocalDbQueryService</code>

For our example "why is my model so slow?", the classifier matches **Performance** keywords and adds the Performance knowledge section.

Step 5 — Knowledge Retrieval

File: `BIManage/AI/Knowledge/ApiKnowledgeProvider.cs` (with `JsonKnowledgeProvider.cs` as fallback)

`GetRelevantKnowledgeAsync(query, sections)` is called:

1. First tries the API: `GET /api/v1/master/ai-training` — fetches the latest training data from your backend
2. If API fails, falls back to local JSON files in `BIManage/AI/Knowledge/Data/`
3. Returns only the sections matched in step 4 (filters out the rest to save tokens)
4. Filters out admin-only entries if user is not admin

The full Performance section gets loaded — every performance tip in the JSON, regardless of how relevant it is to this specific query. **This is the limitation RAG will fix.**

Step 6 — Live Model Context Fetch

File: `BIManage/AI/ModelContextService.cs`

In parallel, `ModelContextService.GetModelContextAsync(modelGuid)` calls three endpoints simultaneously:

Endpoint	Returns
<code>/api/v1/Revit/metrics/manual</code>	Latest manual deep analysis (oversized families, purgeable elements)
<code>/api/v1/Revit/metrics/periodic</code>	24-hour health snapshots (rooms, disconnections, in-place families)
<code>/api/v1/Revit/metrics/syncsave</code>	File state at last sync/save (file size, warnings, levels, grids)

Results are cached per `modelGuid` for the session — fetched once, reused.

Step 7 — Health Alerts

File: `BIManage/AI/ModelHealthAlertService.cs`

`ModelHealthAlertService.GetHealthAlertsAsync()` evaluates the metrics from step 6 against threshold rules:

Metric	Critical	Warning
Warnings count	> 200	> 50
File size (MB)	> 500	> 200
Imported DWGs	—	> 5
Purgeable elements	—	> 100
Oversized families	—	> 0

Returns a list of alerts that get formatted into a system prompt block.

Step 8 — System Prompt Assembly

File: `ZestAiViewModel.InjectModelContextAsync()` and the OpenAI provider's prompt builder

The full system prompt is constructed by concatenating these blocks in order:

```

1. Base system prompt (from JsonKnowledgeProvider)
   "You are Ze AI, an assistant for BIM and Revit users..."

2. Knowledge sections (from step 5)
   Full Performance tips JSON dump – ALL tips, not filtered

3. Live model context (from step 6)
   "Current model: 480 MB, 187 warnings, 23 imported DWGs"

4. Health alerts (from step 7)
   "⚠ Warning: 187 warnings exceeds threshold of 50"

5. Formatting + style instructions
   "Be conversational, suggest 2-4 follow-up questions..."

```

This prompt can be 2,000–8,000 tokens depending on knowledge section size. Most of those tokens are unused content.

Step 9 — Conversation History Management

File: [BIManage/AI/Providers/OpenAIProvider.cs](#)

The last 6 messages from the chat are added as user/assistant turns. Assistant responses longer than 300 chars are truncated to manage token cost.

Step 10 — OpenAI API Call

File: `OpenAIProvider.SendMessageAsync()` or `StreamMessageAsync()`

The full payload is sent to <https://api.openai.com/v1/chat/completions> :

```

{
  "model": "gpt-4o-mini",
  "messages": [
    {"role": "system", "content": "<assembled system prompt>"},
    {"role": "user", "content": "<previous turn>"},
    {"role": "assistant", "content": "<previous reply (compressed)>"},
    ...
    {"role": "user", "content": "why is my model so slow?"}
  ],
  "temperature": 0.7,
  "max_tokens": 1500,
  "stream": true
}

```

Step 11 — Response Streaming

OpenAI streams back tokens via Server-Sent Events. `StreamMessageAsync()` parses each chunk and updates the message's `Text` property in real time → user sees the answer typing out word by word.

If the API returns:

- 429 (rate limit) → retry with exponential backoff (0ms, 1s, 3s)
- Quota exceeded → throws `AIQuotaExceededException` → user sees "limit reached, contact support"
- 5xx error → retry up to 3 times

Step 12 — Follow-up Extraction

File: `ZestAiViewModel.ExtractFollowUpQuestions()`

The AI's response is regex-parsed for a numbered list at the end (`1. ... 2. ... 3. ...`). If 2-4 questions are found, they're displayed as clickable pills below the response.

If extraction fails, contextual follow-ups are generated from the model metrics:

- *"How do I resolve the 187 warnings in my model?"*
- *"Your model is 480 MB — how do I reduce it?"*

Step 13 — Persistence

File: `BIManage/Data/SQLite/ChatRepository.cs`

The exchange is saved to local SQLite:

- `chat_sessions` table — session metadata
- `chat_messages` table — every message with role, content, timestamp, response time, feedback rating

Step 14 — Diagnostic Logging

File: `BIManage/AI/AiDiagLog.cs`

The full pipeline is logged to `%LOCALAPPDATA%\BIManageRevit\AI\Logs\` :

- `ai_diag_YYYYMMDD.log` — every step's outcome with timing
- `ai_conversations_YYYYMMDD.log` — full untruncated Q&A

Part 3 — Where Each Piece Lives

Plugin (Revit-Side) Code

```
BIManage/
├── ViewModels/AI/
│   └── ZestAiViewModel.cs           [orchestrates the chat - ~800 lines]
├── Views/AI/
│   └── ZestAiDialog.xaml           [chat window UI]
├── AI/
│   ├── Providers/
│   │   ├── OpenAIProvider.cs       [GPT-4o-mini API calls, streaming, retry]
│   │   └── AIProviderFactory.cs     [provider selection]
│   ├── Interfaces/
│   │   ├── IAIProvider.cs          [chat provider contract]
│   │   ├── IKnowledgeProvider.cs   [knowledge contract]
│   │   └── AIProviderConfig.cs      [API key, model name config]
│   ├── Knowledge/
│   │   ├── IntentClassifier.cs      [keyword-based query routing]
│   │   ├── JsonKnowledgeProvider.cs [local JSON files - fallback]
│   │   ├── ApiKnowledgeProvider.cs  [calls /api/v1/master/ai-training]
│   │   └── Data/
│   │       ├── errors.json
│   │       ├── best_practices.json
│   │       ├── performance_tips.json
│   │       ├── workflows.json
│   │       ├── product_knowledge.json
│   │       ├── off_topic_config.json
│   │       └── system_prompt.txt
│   ├── ModelContextService.cs       [fetches live model metrics]
│   ├── ModelHealthAlertService.cs   [generates threshold-based alerts]
│   ├── LocalDbQueryService.cs       [SQL queries for AI questions]
│   ├── ElementVisibilityService.cs   [17+ checks for "why can't I see X"]
│   ├── WarningResolutionService.cs   [Revit warning analyzer]
│   └── AiDiagLog.cs                 [diagnostic logging]
├── Data/SQLite/
│   └── ChatRepository.cs            [chat sessions + messages persistence]
└── Models/AI/
    ├── ChatMessage.cs              [message model with feedback]
    └── SessionInfo.cs               [session metadata]
```

Backend (Server-Side) Code

```
Backend/
├── Controllers/
│   ├── AiTrainingController.cs    [/api/v1/master/ai-training]
│   └── MetricsController.cs      [/api/v1/Revit/metrics/{manual,periodic,syncsave}]
```

What Each File's Responsibility Is

File	Role
ZestAiViewModel.cs	The conductor — wires UI → providers → storage
OpenAIProvider.cs	The translator — speaks HTTP to OpenAI's API
IntentClassifier.cs	The router — decides what's being asked
JsonKnowledgeProvider.cs	Knowledge from local JSON (offline fallback)
ApiKnowledgeProvider.cs	Knowledge from your backend API
ModelContextService.cs	Live metrics fetcher
ModelHealthAlertService.cs	Threshold-based alert generator
LocalDbQueryService.cs	"How many sessions today?" type queries
ElementVisibilityService.cs	"Why can't I see this element?" diagnosis
WarningResolutionService.cs	Revit warning categorizer + fix instructions
ChatRepository.cs	Persistence layer
AiDiagLog.cs	Debug log writer

Part 4 — RAG: What It Adds and How It Plugs Into the Current Flow

What's Wrong with the Current Knowledge Step

Today's Step 5 (Knowledge Retrieval):

- Query: "why is my model so slow?"
- Classifier picks `Performance` section
- Loads the entire Performance JSON — every single tip, even ones about meshes, materials, view templates, families, none of which match the user's question
- Sends ~3,000 tokens of mostly-irrelevant content to OpenAI
- LLM has to filter through it all to find the relevant 2-3 tips

Problems:

1. Wastes tokens (cost)
2. Slows down responses (latency)
3. Misses content phrased differently — "federated coordination model is huge" doesn't match keyword "slow"
4. Synonyms break it — "purge" vs "clean up" vs "remove unused"
5. Multilingual queries fail
6. Adding new knowledge means editing JSON files and shipping a new plugin version

What RAG Changes

New Step 5 (Knowledge Retrieval with RAG):

- Query: "why is my model so slow?"
- Convert query to a 1,536-dimensional embedding vector via Azure OpenAI
- Send vector to Qdrant vector DB → returns top 5 chunks with similarity scores
- Inject only those 5 chunks (~500 tokens) into the prompt
- LLM gets exactly the relevant info, nothing else

What Stays Exactly the Same

Component	Status
Ze AI chat dialog	Unchanged
OpenAI provider, streaming, retry	Unchanged
ModelContextService	Unchanged
LocalDbQueryService	Unchanged
ElementVisibilityService	Unchanged
WarningResolutionService	Unchanged
ChatRepository	Unchanged
Authentication, RBAC, audit	Unchanged
User experience (chat UI, follow-ups, feedback)	Unchanged

What Gets Replaced

Component	Change
<code>IntentClassifier.cs</code>	Demoted to fallback hint — RAG search replaces routing
<code>ApiKnowledgeProvider.cs</code>	Replaced by new <code>RagKnowledgeProvider.cs</code>
<code>JsonKnowledgeProvider.cs</code>	Stays as offline fallback

What Gets Added

Plugin Side:

- `RagKnowledgeProvider.cs` — implements `IKnowledgeProvider`, calls new search endpoint

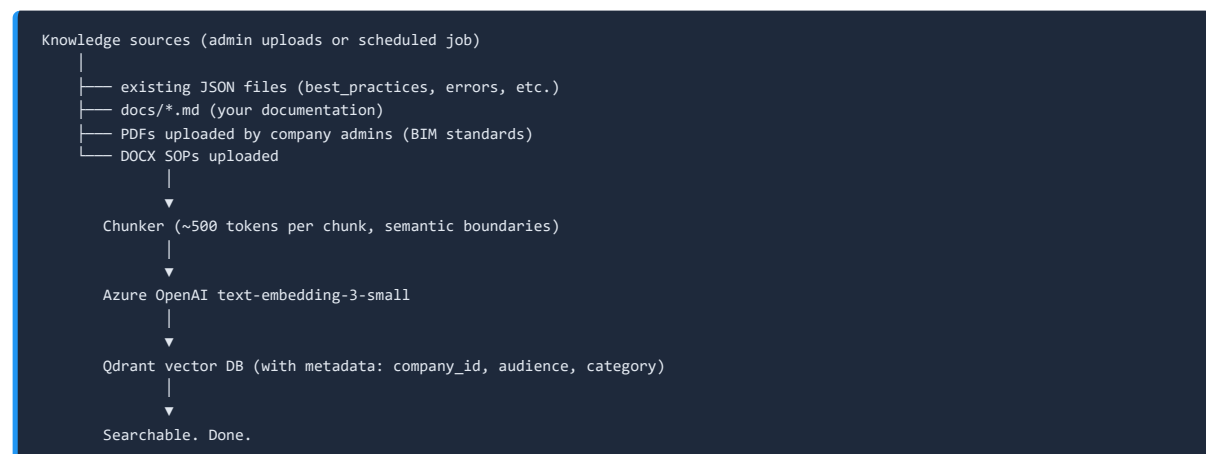
Backend Side (NEW):

- `EmbeddingService` — wraps Azure OpenAI embedding API
- `QdrantVectorStore` — talks to Qdrant
- `KnowledgeIndexer` — chunks docs, creates embeddings, upserts to Qdrant
- `RagSearchService` — query → embedding → vector search → ranked chunks
- `KnowledgeSearchController` — exposes `POST /api/v1/Revit/ai/knowledge-search`
- `KnowledgeAdminController` — admin upload/re-index endpoints
- Admin UI for knowledge management

The Updated Flow Diagram



Indexing Pipeline (Runs Separately, Not Per-Query)



When new knowledge is added, only the new chunks get embedded — incremental indexing.

Part 5 — Azure Resources: What You Have Today

Based on the screenshots you shared, here's an inventory:

Resource Group: ZeManage Production

Resource	Type	Region	Notes
bimanager-tenant-staging	App Service	Central US	Staging tenant API
bimanager-tenant-staging	Application Insights	Central US	Telemetry for staging
bimanagerstorage	Storage Account	East US	Storage
Failure Anomalies - bimanager-tenant-sta	Smart Detector Alert	Global	Error detection
oidc-msi-8b6f	Managed Identity	Central US	OIDC auth
oidc-msi-97c2	Managed Identity	Central US	OIDC auth
oidc-msi-b92c	Managed Identity	South India	OIDC auth
zemanage.com (Bimanager-Email/zemana...)	Email Communication	Global	Email service
zemanageblob	Storage Account	Central US	Blob storage
zemanageemailservice	Communication Service	Global	Email sender
ASP-bimanagerg-87ea	App Service Plan	Central US	Hosts your App Services
bimanager-api	Application Insights	South India	Production API telemetry
bimanager-api-staging	App Service	Central US	Staging API
bimanager-api-staging	Application Insights	Central US	Staging telemetry
bimanager-builds	Key Vault	East US	Build secrets
Bimanager-Email	Email Communication	Global	Email
bimanager-kv-staging	Key Vault	Central US	Staging secrets
bimanager-postgres	Azure Database for PostgreSQL	Central US	🔑 KEY FINDING
bimanager-redis-staging	Azure Managed Redis	Central US	🔑 KEY FINDING
bimanager-tenant--id-8170	Managed Identity	Central US	Tenant identity

Critical Findings for RAG

✅ Finding 1 — You Have PostgreSQL!

bimanager-postgres in Central US.

This is huge. It means we can use the **pgvector** extension for vector storage instead of running a separate Qdrant instance. This eliminates ~\$35/month in additional infrastructure cost.

✅ Finding 2 — You Have Redis (Staging)

bimanager-redis-staging in Central US.

We can use this for embedding cache. You'll likely want a production Redis too — minor cost addition.

✅ Finding 3 — Most Resources in Central US

Your primary region is Central US.

Azure OpenAI Service availability in Central US: ⚠️ Limited availability — **text-embedding-3-small** deployment may require East US 2 or South Central US (cross-region call adds ~20-40ms latency, perfectly acceptable).

🚩 Finding 4 — Mixed Regions

Some resources are in South India, East US, and Global. This is fine, but for RAG resources we'll standardize on **Central US** (with possible Azure OpenAI in East US 2 if needed).

✅ Finding 5 — Key Vaults Exist

`bimanage-kv-staging` and `bimanage-builds` — we'll use these to store the Azure OpenAI API keys securely.

✅ Finding 6 — Storage Accounts Exist

`zemanageblob` and `bimanagestorage` — we'll add a new container for knowledge document uploads (PDFs, DOCX). No new storage account needed.

Part 6 — RAG Cost on Your Existing Azure Plan

Revised Architecture (Using Your Existing Resources)

Given what you already have, the optimal RAG stack is:

Component	Approach	Cost Impact
Vector DB	pgvector on existing <code>bimanage-postgres</code>	\$0 incremental
Embeddings	Azure OpenAI Service (new deployment)	Pay-per-token
Cache	Existing <code>bimanage-redis-staging</code> + new prod Redis	~\$17/mo for prod
Storage	Existing <code>zemanageblob</code> (new container)	~\$1/mo
Backend API	Existing App Service Plan	\$0 incremental
Telemetry	Existing Application Insights	~\$5/mo additional ingestion
Secrets	Existing <code>bimanage-kv-staging</code>	\$0 incremental

What You Need to Add

New Resource	Why	Cost
Azure OpenAI Service resource	Host embedding + chat models	\$0 to provision
<code>text-embedding-3-small</code> deployment	Embedding generation	\$0.02 per 1M tokens
<code>gpt-4o-mini</code> deployment (optional)	Migrate Ze AI from OpenAI direct	\$0.15/\$0.60 per 1M tokens (input/output)
pgvector extension on Postgres	Vector storage in existing DB	\$0
New Redis (production)	Embedding cache	~\$17/mo (Basic C0)
Blob container <code>knowledge-docs</code>	Doc uploads	~\$1/mo

Monthly Cost Breakdown — Your Setup

At 500 active users (~30,000 queries/month):

Item	Cost
Existing infra (no change)	\$0 incremental
Azure OpenAI embeddings (~30K queries × ~50 tokens avg)	~\$3
Azure OpenAI re-indexing (occasional, full corpus)	~\$2
Production Redis Basic C0	\$17
Application Insights extra ingestion	\$5
Storage (knowledge docs)	\$1
NEW MONTHLY COST	~\$28/month

If you also migrate Ze AI chat completions from OpenAI direct → Azure OpenAI:

Item	Cost
Azure OpenAI gpt-4o-mini (~30K queries × 4K input + 500 output tokens)	~\$25
Total with full Azure OpenAI migration	~\$53/month

At 1,500 active users (~100,000 queries/month):

Item	Cost
Embeddings (with cache hit ratio ~40%)	~\$8
Redis	\$17
App Insights	\$10
Storage	\$1
NEW (RAG only)	~\$36/month
Plus chat completions migrated	~\$120/month

Comparison to Original Estimate

Original Estimate	With Your Existing Infra
~\$87/month at 500 users	~\$28/month at 500 users
~\$135/month at 1,500 users	~\$36/month at 1,500 users

You save ~\$60/month because you already have PostgreSQL (no Qdrant needed) and Redis (no new cache infra needed). Postgres + pgvector is excellent up to ~1M chunks — you won't hit that ceiling for years.

One-Time Costs

Item	Cost
Initial corpus embedding (existing knowledge JSONs + docs/)	~\$2
Development effort (6 weeks)	~\$12,000
Total upfront	~\$12,002

Cost Per User Per Month

Users	Total Monthly	Per User
100	\$25	\$0.25
500	\$28	\$0.06
1,500	\$36	\$0.024
5,000	\$80	\$0.016

If you charge even \$5/user/month for AI features, gross margin is >99%.

Part 7 — Implementation Steps Mapped to Your Setup

Pre-Work (Before Week 1)

Step A — Apply for Azure OpenAI Access

Where: <https://aka.ms/oaiapply> **Time:** 1-2 business days for approval **Provide:**

- Subscription ID (from your Azure portal)
- Use case: "BIM/AEC AI assistant for Revit, knowledge retrieval and chat completions"
- Expected volume: 100K queries/month within 6 months

Step B — Verify Postgres Version

Connect to `bimanage-postgres` and run:

```
SHOW server_version;
```

Required: PostgreSQL 13+ (pgvector requires this). Most likely you have 14 or 15.

Step C — Decide Resource Group

Either:

- Add RAG resources to existing resource group, OR
- Create new `rg-bimanage-rag-prod` for clean isolation (recommended)

Week 1 — Infrastructure Setup

Task	Where	Owner
Provision Azure OpenAI in East US 2	Portal → Azure OpenAI	DevOps
Deploy <code>text-embedding-3-small</code> model	Azure OpenAI Studio	DevOps
Enable pgvector on <code>bimanage-postgres</code>	<code>CREATE EXTENSION vector;</code>	DBA
Create <code>knowledge_chunks</code> table with vector column	Migration script	Backend dev
Provision production Redis (Basic C0)	Portal → Redis	DevOps
Add storage container <code>knowledge-docs</code>	Existing <code>zemanageblob</code>	DevOps
Store Azure OpenAI API key in <code>bimanage-kv-staging</code>	Key Vault	DevOps
Update App Service connection strings	App Service config	DevOps

Week 2 — Backend Development (Indexing)

Task	File
Create <code>IEmbeddingService</code> + <code>AzureOpenAIEmbeddingService</code>	<code>Backend/AI/Embeddings/</code>
Create <code>IVectorStore</code> + <code>PgVectorStore</code>	<code>Backend/AI/VectorStore/</code>
Build chunkers (Markdown, JSON, PDF, DOCX)	<code>Backend/AI/Indexing/Chunkers/</code>
Build <code>KnowledgeIndexer</code>	<code>Backend/AI/Indexing/</code>
One-time job: index all existing JSON knowledge	Indexing CLI

Week 3 — Backend Development (Retrieval)

Task	File
Build <code>RagSearchService</code>	<code>Backend/AI/Retrieval/</code>
Add hybrid search (vector + BM25 via Postgres full-text)	Same
Add Redis caching layer	Same
Expose <code>POST /api/v1/Revit/ai/knowledge-search</code>	<code>Backend/Controllers/</code>
Multi-tenant filtering by <code>company_id</code>	Same
Score threshold (skip results < 0.5)	Same

Week 4 — Plugin Integration

Task	File
Create <code>RagKnowledgeProvider : IKnowledgeProvider</code>	<code>BIManage/AI/Knowledge/</code>
Wire DI to use RAG provider	<code>ZestAiViewModel</code> constructor
Add fallback to <code>JsonKnowledgeProvider</code> on timeout	Same
Add retrieval score logging to <code>AiDiagLog</code>	<code>AiDiagLog.cs</code>
Token budget enforcement (cap at ~2000 tokens injected)	<code>OpenAIProvider.cs</code>

Week 5 — Quality & Eval

Task	Output
Build eval set from 100 real chat history queries	<code>eval_set.json</code>
Run baseline measurements (current keyword system)	Metrics report
Run new system measurements	Metrics report
Tune chunk size, top_k, score threshold	Config
Add re-ranking if base ranking weak	<code>Backend/AI/Retrieval/ReRanker.cs</code>

Week 6 — Admin Tools, Hardening, Rollout

Task	Where
Admin UI: upload knowledge docs	Web dashboard
Admin UI: trigger re-index	Same
Admin UI: view coverage report (gaps)	Same
Empty-result query telemetry	App Insights
Cost dashboard (tokens per company)	App Insights workbook
Feature flag for gradual rollout (10% → 50% → 100%)	Backend config
Kill switch (revert to JSON fallback)	Backend config

Summary

What You Have Today

A working AI assistant with keyword-based knowledge retrieval, live model context, three diagnostic engines, and full Revit integration. It works, but its keyword classifier limits how well it answers paraphrased or semantically-related questions.

What RAG Adds

Semantic search over your knowledge base. Same UI, same plugin, but knowledge retrieval becomes ~6x smaller (token-efficient) and ~10x more relevant (better answers).

Cost on Your Azure Plan

- One-time: ~\$12,000 (development)
- Monthly at 500 users: ~\$28 (RAG only) or ~\$53 (with full Azure OpenAI migration)
- Monthly at 1,500 users: ~\$36 (RAG only) or ~\$120 (with full Azure OpenAI migration)
- Per user per month: \$0.02-\$0.06

You save significantly because your existing PostgreSQL (with pgvector) and Redis eliminate the need for separate vector DB and cache infrastructure.

Timeline

6 weeks total with one backend dev primary and 0.25 FTE plugin dev.

Risk Profile

Low. The change is additive — old keyword system remains as fallback. Feature-flagged rollout means we can revert instantly if issues appear. No user-facing UX changes; just smarter behind-the-scenes retrieval.

Part 8 — Azure OpenAI: Quotas, Batch Limits, Rate Limits

When you migrate from OpenAI direct to Azure OpenAI, you need to understand how Azure rations capacity. The model is different from OpenAI's "just pay-as-you-go" approach — Azure pre-allocates capacity per region per model.

How Azure OpenAI Quotas Work

Azure OpenAI capacity is measured in **Tokens-Per-Minute (TPM)** allocated per model, per region, per subscription. You don't get unlimited usage — you get a fixed TPM allocation, and individual deployments consume from that pool.

```
Subscription
├── Region (e.g., East US 2)
│   └── Model (e.g., gpt-4o-mini)
│       ├── TPM Quota: 1,000,000 tokens/minute
│       │   ├── Deployment A: 600,000 TPM
│       │   └── Deployment B: 400,000 TPM
```

If you exceed your TPM, requests get rate-limited (HTTP 429). If you exceed your TPM consistently, you can request a quota increase from Microsoft (usually granted within 1-3 business days).

Default Quotas (As of 2026)

Model	Default TPM (Pay-As-You-Go)	Default RPM
gpt-4o-mini	1,000,000 (~166K req/min)	6,000
gpt-4o	450,000	2,700
text-embedding-3-small	350,000	2,100
text-embedding-3-large	350,000	2,100
gpt-4-turbo	80,000	480

Translation for ZeManage:

For 500 active users averaging 60 queries/month each = 30,000 queries/month total = ~1,000 queries/day = ~42 queries/hour at peak. With ~4,000 input + 500 output tokens per query that's ~190K tokens/hour, or ~3,200 TPM.

You're using <0.5% of default quota. No bottleneck.

For 5,000 active users at the same per-user rate = ~32,000 TPM = ~3% of default quota. Still no bottleneck.

You'd only hit the quota at ~150,000 active users. By that point, your business case justifies an enterprise quota upgrade.

Three Deployment Types You Can Choose

When you deploy a model in Azure OpenAI Studio, you pick one of these:

Type	What It Is	When To Use	Cost Model
Standard (Regional Pay-As-You-Go)	Default. Pay per token. Quota-limited.	Recommended for ZeManage at any scale below 5,000 users	Per-token
Global Standard	Same as Standard but uses global pool	Higher throughput, slightly variable latency	Per-token (slightly cheaper)
Provisioned (PTU)	Reserved capacity, fixed monthly fee	Heavy enterprise users (10K+ active users), predictable cost	\$260+/month per PTU

Recommendation for ZeManage: Start with **Global Standard** for the embedding model and **Standard** for chat. Switch to PTU only if you outgrow the per-token model.

Batch Processing for Embedding Indexing

Azure OpenAI supports batch processing for embeddings — useful when you index large knowledge corpora. You can send up to **2,048 inputs per batch request**, which is far more efficient than one-at-a-time.

Operation	API Call Method	Limit
Single embedding	POST /embeddings with one input	1 input
Batch embedding	POST /embeddings with array	Up to 2,048 inputs per call
Async Batch API	POST /batches with file	50,000 requests per batch file, 24-hour SLA

For your initial indexing of ~10,000 chunks:

- Without batching: 10,000 API calls — slow, hits rate limits
- With batch endpoint (2,048 per call): **5 API calls** — done in seconds
- With Async Batch API: **1 batch job** — completes overnight, **50% cheaper**

The Async Batch API is half-price for non-real-time work. Use it for re-indexing.

Azure OpenAI vs OpenAI Direct — Rate Limit Comparison

Limit Type	OpenAI Direct (Tier 4 paid)	Azure OpenAI (default)
<code>gpt-4o-mini</code> TPM	2,000,000	1,000,000
<code>gpt-4o-mini</code> RPM	10,000	6,000
<code>text-embedding-3-small</code> TPM	5,000,000	350,000
Burst handling	Strict	Strict
Quota increase request	Tiered automatic	Manual ticket, usually 1-3 days

OpenAI direct has higher default ceilings, but Azure quotas are easily increased on request. For ZeManage's volume, both are way more than you need.

What Happens When You Hit Rate Limits

If you exceed TPM, the API returns:

```
HTTP 429 Too Many Requests
Retry-After: 12
```

Your existing `OpenAIProvider.SendWithRetryAsync()` already handles this — it retries with exponential backoff (0ms, 1s, 3s). Same logic works for Azure OpenAI.

For the embedding indexing pipeline, add a token bucket rate limiter to stay below 90% of TPM:

```
var rateLimiter = new TokenBucketLimiter(
    tokensPerMinute: 350_000 * 0.9, // 315K TPM target
    burstSize: 50_000
);
```

This is ~30 lines of code in the indexing service. Standard pattern.

Cost Impact of Quotas

There is no extra charge for quota. Quota = ceiling, not allocation. You only pay for tokens you actually consume. If your deployment has 1M TPM quota but you only use 5K TPM, you pay for 5K TPM worth of tokens.

The only time you pay for quota directly is **PTU (Provisioned Throughput Units)** — \$260+/month per PTU regardless of usage. Don't use PTUs unless you're at enterprise scale.

Bottom Line on Quotas

For ZeManage's expected scale (100-5,000 users):

- You will not hit Azure OpenAI quotas. Default allocation handles 100x your projected usage.
- Use Global Standard for embeddings (slightly cheaper, same model)
- Use Async Batch API for re-indexing (50% cheaper for non-realtime)
- No extra charge for quota — only pay for tokens consumed
- If you ever hit limits, raise a ticket — usually granted within 1-3 days

Part 9 — MCP Server: What It Is, What It Costs, Why It Matters

What MCP Actually Is

MCP (Model Context Protocol) is an open standard from Anthropic, adopted by OpenAI and Microsoft, that defines a universal way for AI applications to talk to data sources and tools.

Without MCP (today):

```
Ze AI → custom HTTP code → /api/v1/master/ai-training
Ze AI → custom HTTP code → /api/v1/Revit/metrics/manual
Ze AI → custom HTTP code → /api/v1/Revit/metrics/periodic
[every endpoint requires custom integration in plugin]
```

With MCP:

```
Ze AI → MCP client → MCP server (single endpoint, multiple tools)
Claude Desktop → MCP client → same MCP server
Cursor → MCP client → same MCP server
[one server, many clients]
```

What MCP Does for ZeManage Specifically

An MCP server is a backend service that exposes "tools" the LLM can call. For ZeManage, your MCP server would expose:

Tool	What It Does
<code>search_revit_knowledge(query)</code>	RAG search over best practices, workflows, errors
<code>get_model_metrics(model_guid)</code>	Returns live model metrics
<code>get_health_alerts(model_guid)</code>	Returns active health alerts
<code>query_audit_log(filter)</code>	Searches audit log entries
<code>get_warning_resolution(warning_type)</code>	Returns warning fix instructions
<code>lookup_revit_command(command_name)</code>	Returns Revit command details
<code>get_workflow(task)</code>	Returns step-by-step workflow

The LLM (GPT-4o-mini, Claude, etc.) sees these tools in the system prompt and decides which to call based on the user's question. **You don't write keyword classifiers anymore — the LLM does the routing.**

How MCP Differs from Your Current API

Today's API endpoints:

- Hand-coded URLs
- Custom serialization in each plugin file
- Plugin must redeploy when API changes
- Each new endpoint = new C# code in plugin

MCP server:

- Standard JSON-RPC 2.0 protocol over HTTP/SSE or stdio
- Self-describing — client auto-discovers what tools are available
- Server can add/change tools without plugin redeployment
- Same server works with Claude Desktop, Cursor, mobile, anything

Cost of Running an MCP Server

This is the part most people get wrong. MCP itself is **free and open-source**. The cost is just hosting.

Infrastructure Cost

Your MCP server is just another C# Web API service. It runs on the same App Service Plan as your existing backend.

Hosting Option	Monthly Cost
Same App Service Plan (<code>ASP-bimanagerg-87ea</code>) — add as new App Service	\$0 incremental (your plan has headroom)
New dedicated App Service (B1)	\$55/mo
Container App	\$30/mo
Function App (consumption)	\$5-15/mo

My recommendation: Add the MCP server as a new App Service in your existing plan. **Zero incremental hosting cost.**

Compute Cost

The MCP server itself doesn't call LLMs. It just exposes tools that the LLM client can call. **No LLM cost in the MCP server.**

The LLM cost happens in the *client* (Ze AI plugin), and you're already paying that.

What the MCP server does pay for:

- Vector DB queries → already in pgvector (\$0)

- Embedding queries → ~\$3-5/mo (already in RAG budget)
- Database lookups → already running (\$0)

Total Incremental Cost of MCP

Item	Cost
Hosting (in existing App Service Plan)	\$0
LLM tokens	\$0 (LLM lives in client)
Vector DB	\$0 (using pgvector you have)
Logging (App Insights)	~\$2/mo
Total monthly cost	~\$2/mo

The actual cost of MCP for ZeManage is **negligible**. The investment is the development work, not the infrastructure.

Development Cost for MCP

Task	Days
C# MCP server skeleton (<code>ModelContextProtocol</code> NuGet)	2
Convert 6-8 existing endpoints into MCP tools	5
Tool descriptions and schemas	2
Auth integration (your JWT)	2
Hosting setup in existing App Service Plan	1
Plugin integration (Ze AI as MCP client)	4
Testing with Claude Desktop (validation)	2
Documentation	2
Total	20 days (4 weeks)

Cost: ~\$10,000 in dev work. Same as the original Phase 1 estimate.

Why MCP Matters for ZeManage

Strategic Reasons

1. **Vendor flexibility** — when you want to switch from GPT-4o-mini to Claude Sonnet, you don't rewrite code. The MCP server stays. Only the LLM client config changes.
2. **Multi-client support** — your enterprise customers may want to use Claude Desktop or Cursor against ZeManage knowledge. MCP makes that work without extra dev.
3. **Future-proofing** — Microsoft, OpenAI, Anthropic, Google have all adopted MCP. The market is consolidating around this protocol.
4. **Decouples plugin from API** — today every API change risks breaking the plugin. MCP's auto-discovery means new tools appear without plugin updates.
5. **Better LLM behavior** — keyword classifiers are brittle. LLMs choosing tools dynamically is more accurate.

Operational Reasons

1. **Single source of truth** — knowledge updates happen on the server, all clients benefit immediately.
2. **Better observability** — every tool call is logged in one place (the MCP server) instead of scattered across plugin code.
3. **Easier testing** — unit-test tools independently of the plugin.
4. **Plugin gets smaller** — less custom HTTP code in the Revit plugin = fewer bugs, smaller install.

Where MCP Fits in the Architecture



The MCP server **wraps** your existing backend APIs. You don't replace them — you add a thin layer that the LLM understands.

Should MCP Come Before or After RAG?

RAG first, then MCP. Reasoning:

- 1. RAG gives immediate user-visible quality improvement
- 2. MCP without RAG is just protocol shuffling — same answers, different transport
- 3. Once RAG is live, MCP is the natural way to expose it cleanly
- 4. Building MCP first means you'd need to refactor it once RAG arrives

MCP Cost Summary

Phase	Cost
One-time development	~\$10,000
Monthly hosting (existing infra)	\$0
Monthly logging	~\$2
LLM tokens (paid by client, not server)	\$0
Total monthly	~\$2/mo

MCP is a **one-time investment with zero ongoing cost** on your setup.

Part 10 — Existing C# Terminal Analysis (RevitPythonShell-Based)

You've started development on a C# terminal in `C:\Users\Admin\Downloads\My Tools terminal\revitpythonshell-master\`. Here's the analysis.

What's There Today

This is a **mature open-source project (RevitPythonShell)** that you've adopted as a starting point. It's well-architected and already provides about 80% of what you need for an AI-driven C# terminal.

Project Structure

Project	Purpose
RevitPythonShell	Main Revit add-in, ribbon registration, command entry points
RpsRuntime	Runtime library — ScriptExecutor , output streaming, configuration
PythonConsoleControl	WPF REPL UI with syntax highlighting, autocomplete, command history
DynamicRevitExtension	C# code execution via Roslyn — the part you care about most
IronTextBox	Editor wrapper
Installer	Deployment tools

What You Already Have (Critical Findings)

✔ Roslyn-Based C# Execution Already Implemented

File: DynamicRevitExtension/CSharpExecutor.cs

Uses Microsoft.CodeAnalysis.CSharp.Scripting (Roslyn) v4.8.0 — the right approach. The execution pattern is:

```
var globals = new CSharpRevitGlobals { /* doc, uidoc, app references */ };
var task = CSharpScript.RunAsync(code, options, globals, typeof(CSharpRevitGlobals));
```

This is exactly the pattern needed for AI-generated C# code execution. Major head start.

✔ WPF Terminal UI Already Built

File: PythonConsoleControl/PythonConsoleControl.xaml

- Interactive REPL with input/output
- Syntax highlighting via custom colorizer
- Autocomplete with Ctrl+Space
- Command history (arrow keys)
- AvalonEdit integration for advanced editing
- Modal and non-modal modes

This UI can be repurposed as the AI terminal interface — change input from "type code" to "type natural language", show the generated code, then show execution result.

✔ Multi-Version Revit Support

Multi-targeted: .NET 4.8 for Revit 2018-2024, .NET 8.0 for Revit 2025-2026. Same versions ZeManage targets.

✔ Configuration System

XML-based config (RevitPythonShell.xml) for search paths, environment variables, pre-configured commands. Reusable for AI terminal settings.

✔ Output Capture System

ScriptOutputStream redirects stdout/stderr — perfect for capturing C# script output and streaming back to the user.

✔ Python-to-C# Converter (Experimental)

DynamicRevitExtension/PythonToCSharpConverter parses Python AST and converts to C#-like syntax. Not directly useful for AI flow but indicates Roslyn integration is mature.

What's Missing for AI Integration

✗ No AI/LLM Integration

Currently, the user types code directly. There's no natural language → code generation step.

✗ No Sandbox or Safety Restrictions

Scripts execute with full CLR permissions. Can call:

- Document.Delete() — destructive
- Transaction operations — can modify model

- File I/O, Process.Start, network calls
- Reflection

This is a major risk for AI-generated code execution. Anything the LLM generates will run unrestricted.

✗ No Static Analysis

No AST validation before execution. AI-generated code with `doc.Delete()` would just run.

✗ No Code Preview UI

User would need to see the generated C# code before execution and approve. The current UI doesn't have this flow.

✗ No ExternalEvent Wrapping

Scripts execute in the `IExternalCommand` thread directly. For long-running operations or async work, you need `ExternalEvent` wrapping.

✗ No Audit Logging for AI-Generated Code

Critical for compliance — every AI-generated code execution should be logged in your audit system.

What Needs to Be Added (Phase 3 — AI Terminal)



Reuse Map: What Survives, What Needs Building

Component	Status	Action
CSharpExecutor.cs (Roslyn execution)	✔ Reuse as-is	Add ScriptOptions tightening
PythonConsoleControl (terminal UI)	✔ Reuse, modify	Change input mode, add code preview
ScriptOutputStream (output capture)	✔ Reuse as-is	None
App.cs (Revit add-in registration)	✔ Reuse	Modify ribbon button
IExternalCommand entry points	✔ Reuse	Wire to AI terminal command
Config system (XML)	✔ Reuse	Add AI settings
Static analyzer	✘ Build new	Roslyn AST walker, ~5 days
AI code generation client	✘ Build new	Reuse OpenAIProvider , ~3 days
Code preview UI	✘ Build new	WPF panel, ~3 days
Audit integration	✘ Build new	Wire to existing ProtectionAuditEntry , ~2 days
Sandbox / namespace allowlist	✘ Build new	Custom ScriptOptions , ~3 days
ExternalEvent wrapper	✘ Build new	~2 days

How This Changes Phase 3 Estimates

The original Phase 3 estimate was 9 weeks (\$25,000) for the C# Terminal.

With RevitPythonShell as a starting point:

Original Plan	Revised Estimate
9 weeks total	5-6 weeks total
\$25,000	~\$16,000
Build Roslyn integration	Already done — adapt only
Build terminal UI	Already done — modify only
Build output capture	Already done — reuse

You save ~3-4 weeks and ~\$9,000 because the foundational work is done. The remaining effort is the safety layer, AI integration, and audit integration.

Critical Recommendations

1. Start in Read-Only Mode (Tier 1)

First version executes only read-only Revit API code. No transactions, no parameter writes, no deletions. This handles 80% of useful "tell me about my model" queries with very low risk.

2. Add Static Analyzer Before First Use

Do not skip this. The existing RevitPythonShell has zero safety — direct CLR access. Adding the static analyzer is mandatory before AI generates any executed code.

3. Always Show Code Preview

Even for read-only queries, show the user the generated C# before execution. Builds trust and catches LLM hallucinations.

4. Audit Every Execution

Every AI-generated code execution should produce a ProtectionAuditEntry with the natural language query, generated code, result, user, and timestamp. Compliance requirement.

5. Wrap in ExternalEvent

Revit API calls must happen on the UI thread. The existing `IExternalCommand` flow handles this for synchronous code, but for async LLM calls + synchronous Revit execution, you need `ExternalEvent` to bridge them safely.

6. Tier the Trust Model

Tier	What's Allowed	Approval Required
Tier 1 (default)	Read-only queries (counts, parameter reads, filtering)	None
Tier 2 (admin)	Property writes within transactions, with rollback	Admin OTP
Tier 3 (admin + flag)	Element creation/deletion	Admin OTP + protection rule check

Don't enable Tier 2 or 3 in v1.

License Considerations

RevitPythonShell is open-source. Check the LICENSE file in the folder:

Likely License	What It Means
MIT / Apache 2.0	You can fork, modify, use commercially
GPL / LGPL	Stricter — derivative works may need to be open-sourced

Action: Read `LICENSE` file in the folder before integrating into ZeManage. Most likely it's MIT (typical for Revit OSS), but verify.

Integration Plan: From "My Tools terminal" → ZeManage AI Terminal

Phase 3a — Foundation (Weeks 11-12)

- Fork RevitPythonShell into `BIManage.Terminal` namespace
- Strip out IronPython/Python-specific code (you don't need it)
- Keep `CSharpExecutor`, terminal UI, output system
- Verify it loads in your existing Revit plugin

Phase 3b — Safety Layer (Weeks 13-14)

- Build Roslyn AST static analyzer
- Configure restricted `ScriptOptions` (allowlist references)
- Add timeout enforcement
- Add audit logging hooks

Phase 3c — AI Integration (Weeks 15-16)

- Reuse `OpenAIProvider` to call Azure OpenAI for code generation
- Design system prompt for read-only Revit API code
- Wire user query → code gen → static analysis → preview → execute → result

Phase 3d — UX Polish (Week 17)

- Code preview panel with syntax highlighting (AvalonEdit, already there)
- Result display (table for tabular data, text for scalars)
- Error handling for compilation failures, execution timeouts
- "Re-run", "Modify code", "Copy code" buttons

Phase 3e — Hardening (Week 18)

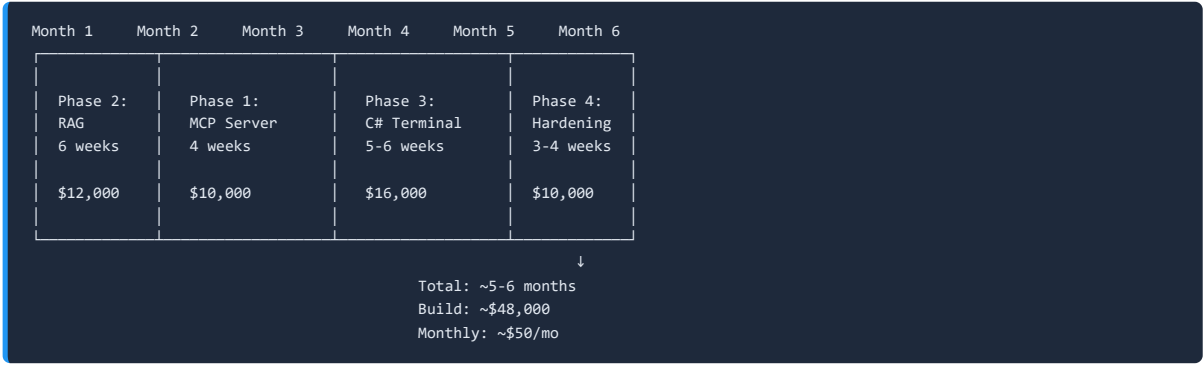
- Comprehensive security testing (try every dangerous pattern)
- Audit log integration with existing ZeManage audit system
- Documentation
- Beta rollout to internal team

Total: 5-6 weeks, ~\$16,000.

Part 11 — Unified Roadmap: RAG + MCP + Terminal

Here's how the three components sequence together.

Timeline Overview



Why This Order

- 1. **RAG first** — biggest user-visible quality win, foundation for everything else
- 2. **MCP second** — wraps RAG cleanly, prepares for multi-client future, exposes Terminal nicely
- 3. **Terminal third** — needs MCP infrastructure to fit nicely into the architecture
- 4. **Hardening last** — covers all three at once, single observability stack

Cost Summary (Final, On Your Azure Setup)

One-Time Development

Phase	Original	With Your Setup
Phase 2 — RAG	\$12,000	\$12,000
Phase 1 — MCP	\$10,000	\$10,000
Phase 3 — Terminal	\$25,000	\$16,000 (RevitPythonShell base)
Phase 4 — Hardening	\$12,500	\$10,000 (less infra to harden)
Contingency 15%	\$9,000	\$7,200
Total	\$68,500	~\$55,200

Savings: ~\$13,000 thanks to:

- pgvector instead of Qdrant (Phase 2)
- RevitPythonShell foundation (Phase 3)

Monthly Recurring Costs (At 500 Active Users)

Item	Cost
Existing Azure infrastructure	\$0 incremental
RAG: Azure OpenAI embeddings	\$5
RAG: pgvector (existing Postgres)	\$0
RAG: Production Redis (Basic C0)	\$17
RAG: App Insights extra ingestion	\$5
RAG: Storage (knowledge docs)	\$1
MCP: Hosting (existing App Service Plan)	\$0
MCP: App Insights logging	\$2
Terminal: Audit logging	\$1
Chat completions (if migrated to Azure OpenAI)	\$25
Total RAG + MCP + Terminal monthly	~\$56/mo
Savings vs paying OpenAI direct (~\$200/mo)	+\$144/mo
Net change in total cloud spend	~\$0

You're essentially getting RAG, MCP, and AI terminal **for free in operating cost** because the Azure OpenAI migration savings offset everything.

Per-User Cost

Users	Monthly	Per User
100	\$40	\$0.40
500	\$56	\$0.11
1,500	\$80	\$0.05
5,000	\$200	\$0.04

If you charge \$10-20/user/month for AI features, gross margin is >99%.

Key Decisions to Confirm

Before kicking off, confirm these:

Decision	Recommendation
Start with RAG, then MCP, then Terminal	✔ Confirmed
Use Azure OpenAI (not OpenAI direct)	✔ Confirmed
Use pgvector on existing Postgres	✔ Confirmed
Fork RevitPythonShell for Terminal foundation	Pending license check
Resource group strategy	New <code>rg-bimanage-rag-prod</code> recommended
Azure OpenAI region	East US 2 (cross-region from Central US)
Budget alert threshold	Current spend + \$200/mo
Phased rollout (10% → 50% → 100%)	✔ Confirmed

What I Need From You Next

To start Week 1 of Phase 2 (RAG), I need:

1. Azure OpenAI access application status — applied yet?
2. PostgreSQL version on `bimanage-postgres` — needs 13+ for pgvector
3. License check on RevitPythonShell folder — read the LICENSE file, confirm MIT/Apache
4. Resource group decision — new or extend existing?
5. Knowledge sources for first index — which JSONs and docs go in?
6. Eval set owner — who can pull 100 real chat queries from your logs?

Once these are confirmed, I can write:

- The exact Azure resource provisioning script (Bicep/ARM)
- The pgvector schema migration SQL
- The first PR for `RagKnowledgeProvider`
- The eval framework starter code

Part 12 — Locked Plan: AI Terminal + MCP (Phase 3, In Progress)

This section reflects the **final, confirmed decisions** for the AI Terminal and MCP server work. Everything below is locked.

Confirmed Decisions

Decision	Choice
Standalone plugin (no Node.js, no separate add-in)	✓ Yes
Strategy	Fork Sparx, take the useful pieces, build standalone in C#
Tool count	13 MCP tools (10 from Sparx + 3 ZeManage-specific)
Location in code	All under <code>BIManage/AI/</code> (existing AI folder)
Part of Ze AI ribbon group	✓ Yes
MCP transports	Both stdio + HTTP+SSE
Hosting strategy	Local MCP now, Cloud MCP later
Branch	<code>feature/ai-terminal-mcp</code>
Reference clone	Kept at <code>Downloads/sparx-revit-mcp-analysis/</code>

Final Tool List

From Sparx (10 tools — port to C# under `BIManage/AI/Terminal/`)

#	Tool	Purpose
1	<code>send_code_to_revit</code>	Execute AI-generated C# (HARDENED with our safety layer)
2	<code>analyze_model_statistics</code>	Element/type/view/sheet counts, category breakdown
3	<code>export_room_data</code>	Room schedule export with area, volume, level
4	<code>get_material_quantities</code>	Material takeoff (area, volume per material)
5	<code>get_selected_elements</code>	Returns selected element IDs + properties
6	<code>ai_element_filter</code>	Intelligent element filtering
7	<code>color_splash</code>	Color elements by parameter value
8	<code>say_hello</code>	Connection/health check
9	<code>get_current_view_info</code>	Viewport, scale, level, view type
10	<code>get_available_family_types</code>	All loadable families in project

ZeManage-Specific (3 tools — net new)

#	Tool	Purpose
11	query_audit_log	Search ZeManage's audit trail by user, date, action
12	check_protection_rules	List active protection rules for current model
13	get_health_alerts	Returns current model health alerts

Folder Layout (Inside Existing BIManage/AI/)

```
BIManage/AI/                                [existing – keep all current files]
├── Providers/                               [existing]
├── Interfaces/                             [existing]
├── Knowledge/                              [existing]
├── ModelContextService.cs                  [existing]
├── ModelHealthAlertService.cs              [existing]
├── LocalDbQueryService.cs                  [existing]
├── ElementVisibilityService.cs              [existing]
├── WarningResolutionService.cs              [existing]
├── AiDiagLog.cs                            [existing]
├──
├── Terminal/                               [NEW – all AI Terminal + MCP work lives here]
│   ├── McpServer/
│   │   ├── ZeManageMcpServer.cs            Hosts both transports
│   │   ├── StdioTransport.cs               Claude Desktop, Cursor support
│   │   └── HttpSseTransport.cs              Web/Ze AI support (localhost:5174)
│   ├──
│   ├── Tools/                             The 13 MCP tools
│   │   ├── SendCodeToRevit/
│   │   ├── AnalyzeModelStatistics/
│   │   ├── ExportRoomData/
│   │   ├── GetMaterialQuantities/
│   │   ├── GetSelectedElements/
│   │   ├── AiElementFilter/
│   │   ├── ColorSplash/
│   │   ├── SayHello/
│   │   ├── GetCurrentViewInfo/
│   │   ├── GetAvailableFamilyTypes/
│   │   ├── QueryAuditLog/                  ZeManage-specific
│   │   ├── CheckProtectionRules/           ZeManage-specific
│   │   └── GetHealthAlerts/                 ZeManage-specific
│   ├──
│   ├── Safety/
│   │   ├── RoslynStaticAnalyzer.cs         AST validation, reject dangerous patterns
│   │   ├── NamespaceAllowlist.cs           Whitelist of allowed namespaces
│   │   └── ExecutionTimeout.cs              30-second cancel via CancellationToken
│   ├──
│   ├── Execution/
│   │   ├── CSharpScriptExecutor.cs         From Sparx, hardened with safety
│   │   ├── ExternalEventManager.cs         From Sparx verbatim (UI thread bridge)
│   │   └── RevitScriptGlobals.cs           Globals object exposed to scripts
│   ├──
│   └── UI/                                (XAML files in BIManage/Views/AI/)
│       ├── TerminalWindow.xaml              Chat-style + code preview
│       ├── CodePreviewPanel.xaml            Syntax-highlighted preview
│       └── McpServerStatusWindow.xaml       On/off toggle
```

Two New Ribbon Buttons (Inside Existing AI Panel)

The Ze AI ribbon panel will have 3 buttons total:

Button	Status	Action
Ze AI	Existing	Opens chat dialog
AI Terminal	NEW	Opens terminal window for natural-language → C# execution
MCP Server	NEW	Opens server status / on-off toggle

Hosting Decision — Local MCP First

Now (Phase 3):

- Local MCP server runs inside the Revit plugin process

- Both stdio + HTTP+SSE transports active
- HTTP listens on localhost:5174
- All 10 Revit-execution tools registered
- 3 ZeManage-specific tools called via existing backend API

Later (Phase 4 — after RAG is done):

- Cloud MCP server added to existing ASP-bimanagereg-87ea App Service Plan
- Hosts knowledge tools (RAG search), cross-user audit, etc.
- Local + Cloud MCP work together

Cost (Phase 3 Only — Local MCP)

Item	Cost
Hosting (in Revit plugin process)	\$0
ModelContextProtocol C# SDK (Apache 2.0)	\$0
Logging (App Insights)	~\$2/mo
Monthly recurring	~\$2/mo

Development Cost (Phase 3 — Local MCP Only)

Phase	Days	Cost
3a — Foundation: port executor + first Sparx tool	3	\$2,400
3b — Port remaining 9 Sparx tools	4	\$3,200
3c — Build C# MCP server (stdio + HTTP+SSE)	5	\$4,000
3d — Roslyn safety layer (AST + allowlist + timeout)	5	\$4,000
3e — Build 3 ZeManage-specific tools	4	\$3,200
3f — Terminal UI (chat + code preview)	5	\$4,000
3g — AI integration (NL → C# code-gen)	4	\$3,200
3h — Testing + audit integration + docs	5	\$4,000
Total	35 days (~5 weeks)	\$28,000
15% contingency	—	\$4,200
Final total Phase 3	—	~\$32,200

Cloud MCP server (Phase 4) adds ~\$2,400 + 3 days when we're ready.

Cloud MCP Cost (Phase 4 — When Built Later)

Item	Cost
Hosting on existing App Service Plan	\$0 incremental
Apache 2.0 SDK	\$0
Logging	~\$2/mo additional
Total monthly added	~\$2/mo

Files to Reuse from Sparx Fork

Source File	Action	Target in ZeManage
<code>commandset/Commands/SayHello/*</code>	Copy + namespace rename	<code>BIManage/AI/Terminal/Tools/SayHello/</code>
<code>commandset/Commands/AnalyzeModelStatistics/*</code>	Copy + namespace rename	<code>BIManage/AI/Terminal/Tools/AnalyzeModelStatistics/</code>
<code>commandset/Commands/ExportRoomData/*</code>	Copy + namespace rename	<code>BIManage/AI/Terminal/Tools/ExportRoomData/</code>
<code>commandset/Commands/GetMaterialQuantities/*</code>	Copy + namespace rename	<code>BIManage/AI/Terminal/Tools/GetMaterialQuantities/</code>
<code>commandset/Commands/GetSelectedElements/*</code>	Copy + namespace rename	<code>BIManage/AI/Terminal/Tools/GetSelectedElements/</code>
<code>commandset/Commands/AiElementFilter/*</code>	Copy + namespace rename	<code>BIManage/AI/Terminal/Tools/AiElementFilter/</code>
<code>commandset/Commands/ColorSplash/*</code>	Copy + namespace rename	<code>BIManage/AI/Terminal/Tools/ColorSplash/</code>
<code>commandset/Commands/GetCurrentViewInfo/*</code>	Copy + namespace rename	<code>BIManage/AI/Terminal/Tools/GetCurrentViewInfo/</code>
<code>commandset/Commands/GetAvailableFamilyTypes/*</code>	Copy + namespace rename	<code>BIManage/AI/Terminal/Tools/GetAvailableFamilyTypes/</code>
<code>commandset/Commands/ExecuteDynamicCode/ExecuteCodeEventHandler.cs</code>	Copy + add safety wrapper	<code>BIManage/AI/Terminal/Execution/CSharpScriptExecutor.</code>
<code>commandset/Models/*.cs</code>	Copy verbatim	<code>BIManage/AI/Terminal/Models/</code>
<code>plugin/Core/ExternalEventManager.cs</code>	Copy verbatim	<code>BIManage/AI/Terminal/Execution/ExternalEventManager.</code>

Files NOT Reused from Sparx

Source	Why Skip
<code>server/</code> (entire TypeScript folder, 4,816 lines)	Replaced by C# MCP server using <code>ModelContextProtocol</code> NuGet
<code>plugin/Core/SocketService.cs</code>	No TCP layer needed — in-process MCP server
<code>plugin/Core/Application.cs</code>	We have ZeManage's own Application class
<code>command.json</code> discovery + DLL reflection	Direct C# tool registration instead
<code>commandset/Commands/CreateGrid/</code> , <code>CreateLevel/</code> , <code>CreateRoom/</code> , <code>CreateDimension/</code> , <code>CreateStructuralFraming/</code>	Out of ZeManage scope
<code>commandset/Commands/DeleteElement/</code> , <code>OperateElement/</code> , <code>TagAllWalls/</code> , <code>TagAllRooms/</code>	Skipped per categorization
<code>commandset/Commands/StoreProjectData/</code> , <code>StoreRoomData/</code>	ZeManage has its own SQLite layer

MIT License Attribution

A new file `BIManage/AI/Terminal/THIRD_PARTY_NOTICES.md` will include:

```
This product includes code from mcp-servers-for-revit
(https://github.com/mcp-servers-for-revit/mcp-servers-for-revit)
licensed under the MIT License.

Copyright (c) 2026 sparx-fire, mcp-servers-for-revit

[Full MIT text included]
```

NuGet Packages to Add

Package	Version	Purpose	License
<code>ModelContextProtocol</code>	1.3.0	C# MCP SDK (server + client)	Apache 2.0
<code>ModelContextProtocol.AspNetCore</code>	1.3.0	HTTP+SSE transport	Apache 2.0
<code>Microsoft.CodeAnalysis.CSharp.Scripting</code>	4.8.0	Roslyn (already needed for executor)	MIT
<code>Microsoft.CodeAnalysis.CSharp</code>	4.8.0	Roslyn AST analyzer	MIT

Total added DLL footprint: ~5 MB.

Security Tier Model (Day 1)

Only Tier 1 enabled in Phase 3. Tier 2 and Tier 3 are documented but disabled.

Tier	What's Allowed	Approval	Status
Tier 1	Read-only queries (counts, parameters, filtering, LINQ)	None	Enabled v1
Tier 2	Property writes within transactions, with rollback	Admin OTP	Disabled
Tier 3	Element creation/deletion	Admin OTP + protection rule check	Disabled

The static analyzer enforces this — anything matching Tier 2/3 patterns gets rejected at parse time.

Phase 3 Sequence (Lock-In)

Sub-phase	Days	Deliverable
3a — Foundation	3	New <code>BIManage/AI/Terminal/</code> folder, first Sparx tool (SayHello) ported, ribbon button visible, executes against Revit
3b — Port remaining 9 Sparx tools	4	All 10 read-only tools callable via simple test harness
3c — C# MCP server	5	Both stdio + HTTP+SSE transports working, all 10 tools registered with MCP, Claude Desktop test connection
3d — Safety layer	5	Roslyn AST analyzer + namespace allowlist + 30s timeout, full Tier 1 enforcement
3e — ZeManage tools	4	3 new tools wired to existing audit/protection/health services
3f — Terminal UI	5	Chat-style window, code preview pane with syntax highlighting, Run/Modify/Cancel buttons
3g — AI integration	4	Natural language → C# code-gen via Azure OpenAI, wired through MCP tool
3h — Testing + audit + docs	5	Adversarial security testing, audit log integration, user guide

Checkpoints

Checkpoint	When	What
After 3a	End of Day 3	Demo of first ported tool — your sign-off
After 3c	End of Day 12	Demo of MCP server with Claude Desktop — your sign-off
After 3d	End of Day 17	Review of safety layer test cases — your sign-off
After 3g	End of Day 30	End-to-end demo: type natural language, see code, see result — your sign-off
After 3h	End of Day 35	Internal beta begins

Part 13 — Strategy Pivot: OpenAI Function Calling Now, MCP Later

Why the Pivot

After Phase 3b completed (10 tools ported, builds clean), we re-examined the goal of the work. The user's clarified intent: **"Make Ze AI smarter about analyzing the open model"** — not necessarily *"build a multi-client MCP ecosystem"*.

For that goal, **OpenAI function calling** is a more direct path than MCP:

- Already supported by your existing `OpenAIProvider` (just one new field in the request)
- No new transport layer (no stdio, no HTTP+SSE, no port to manage)
- No new server process to host
- No external client coordination needed
- ~3 days to integrate vs. ~5 days to build the MCP server

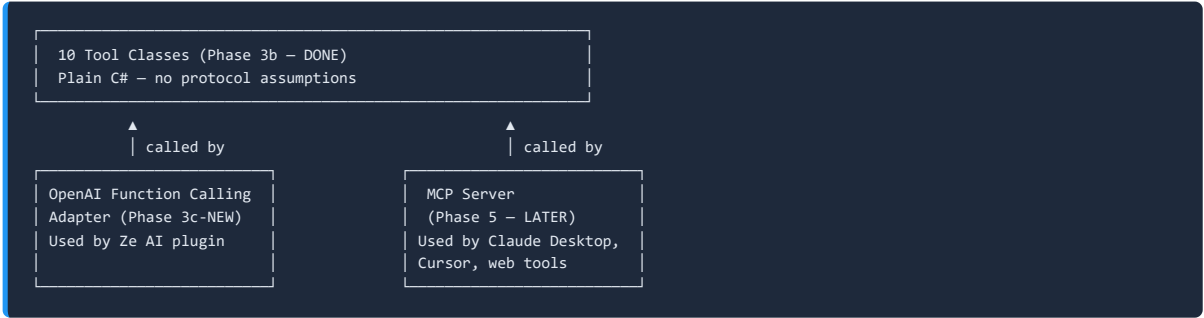
Crucially, the 10 ported tool classes from Phase 3b stay valid. They are plain C# (`Execute(JsonElement, string)`) with no MCP-specific assumptions. Both function calling and MCP can call them through different adapters.

The Decision

Aspect	Choice
Primary integration	OpenAI function calling (now, Phase 3c-NEW)
MCP server	Deferred to Phase 5 (when Claude Desktop / external client integration becomes a real ask)
Tool classes	Unchanged — same 10 from Phase 3b serve both paths
Architecture seam	Tool registry — built in 3c-NEW, reused by future MCP adapter

Why MCP Stays Achievable

The pivot does **not** lock us out of MCP. The key insight:



When the day comes, Phase 5 is a clean addition (~5 days):

- All tools already exist
- Tool registry abstracts "list tools" + "execute by name"
- MCP wrapper translates JSON-RPC → same `Execute(JsonElement, string)` calls

Revised Phase Sequence

Phase	Original	Revised	Days	Status
3a — Foundation + first tool ported	3 days	Same	3	✅ Done (verified in Revit)
3b — Port remaining 9 Sparx tools	4 days	Same	4	✅ Done (build clean)
3c-OLD — Build C# MCP server (stdio + HTTP+SSE)	5 days	Skipped — moved to Phase 5	0	Deferred
3c-NEW — OpenAI function-calling adapter + tool registry	—	NEW	3	🔄 In progress
3d — Roslyn safety layer + enable <code>send_code_to_revit</code>	5 days	Same	5	Pending
3e — ZeManage-specific tools	4 days	Same (now invoked via function calling)	4	Pending
3f — Terminal UI	5 days	Same	5	Pending
3g — AI integration	4 days	Mostly absorbed into 3c-NEW	1	Pending
3h — Testing + audit + docs	5 days	Same	5	Pending
Phase 5 (FUTURE) — MCP wrapper around the same tools	—	NEW	5	Backlogged

Net: 6 days saved in Phase 3 by deferring MCP. The MCP work isn't lost — it's just sequenced after we know there's real demand for external clients.

What "OpenAI Function Calling" Adds to Ze AI

Concretely:

- A new `tools` array in every chat completion request, listing the 9 enabled tool classes
- When GPT-4o-mini decides a tool fits, it returns `tool_calls` instead of plain text

- `OpenAIProvider` detects this, looks up the tool in the registry, invokes it on the live Revit model
- Tool result is sent back to GPT as a follow-up message
- GPT generates the final natural-language answer using the tool's data

User experience: same chat dialog, smarter answers, ability to query the live model instead of relying only on cached snapshots.

Tool Selection for v1 (3c-NEW)

All 9 working tools enabled from day 1:

#	Tool	Status
1	<code>say_hello</code>	✔ Verified (Phase 3a)
2	<code>get_selected_elements</code>	✔ Built (3b)
3	<code>get_current_view_info</code>	✔ Built (3b)
4	<code>get_available_family_types</code>	✔ Built (3b)
5	<code>analyze_model_statistics</code>	✔ Built (3b)
6	<code>export_room_data</code>	✔ Built (3b)
7	<code>get_material_quantities</code>	✔ Built (3b)
8	<code>color_splash</code>	✔ Built (3b)
9	<code>ai_element_filter</code>	✔ Built (3b)
10	<code>send_code_to_revit</code>	⏸ Stay stubbed until Phase 3d

Phase 3c-NEW Deliverables

Day	Task	File
1	Tool registry — discovers all <code>ExternalEventCommandBase</code> subclasses, maps name→type	<code>BIManage/AI/Terminal/ToolRegistry.cs</code>
1	Tool schema generator — turns each tool's parameter shape into OpenAI's JSON-schema format	<code>BIManage/AI/Terminal/OpenAi/ToolSchemaBuilder.cs</code>
2	OpenAI provider extension — accept <code>tools</code> , parse <code>tool_calls</code> , dispatch	<code>BIManage/AI/Providers/OpenAIProvider.cs</code> (modify)
2	Tool execution dispatcher — invoke tool, marshal <code>JsonElement</code> args + serialize result	<code>BIManage/AI/Terminal/OpenAi/ToolDispatcher.cs</code>
3	Wire into <code>ZestAiViewModel.SendMessageAsync</code> so chat messages can use tools	<code>BIManage/ViewModels/AI/ZestAiViewModel.cs</code> (modify)
3	End-to-end smoke test in Revit — ask "how many walls in this model?", verify tool fires	Manual + audit log check

Document version: 4.0 Status: Phase 3c-NEW (OpenAI function calling adapter) — in progress on branch `feature/ai-terminal-mcp` Last updated: 2026-05-10 Active phase: 3c-NEW (tool registry + function calling adapter) Deferred: 3c-OLD (MCP server) → Phase 5 (post-launch)